



Introduction to Database Systems

Agenda

- ▶ **Basic Definitions**
- ▶ **Introduction to DBMS**
- ▶ **Database Users and Environment**
- ▶ **Database Architecture**
- ▶ **Core Database Concepts**
- ▶ **History & Modern Trends**



Basic Definitions

What is Data?

- ▶ **Data** is the raw facts and figures that are collected, stored, and processed to generate meaningful information.
- ▶ **Examples**
 - ▶ **Numbers** (e.g., 25, 3.14)
 - ▶ **Text** (e.g., "Karachi", "Hello")
 - ▶ **Images** (e.g., a photo of a car)
 - ▶ **Audio/Video** (e.g., a recorded lecture, a movie)
 - ▶ **Symbols** (e.g., \$, %, @)



What is Information?

- ▶ Information is the processed, organized, or structured form of data that carries meaning and can be used for decision-making.
- ▶ Examples
 - ▶ Data (Raw): 85, 90, 78, 95
 - ▶ Information (Processed): "The student's average score is 87, which means he has passed with good marks."

Data Vs Information

DATA

INFORMATION

a person's
phone number
(555)123-7798

5551237798



Data -> Information -> Knowledge

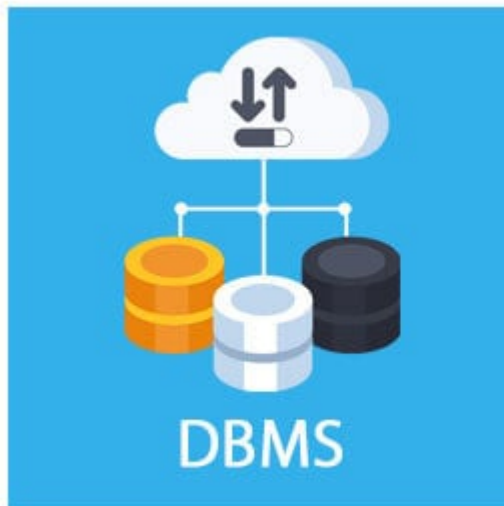


Data -> Information -> Knowledge

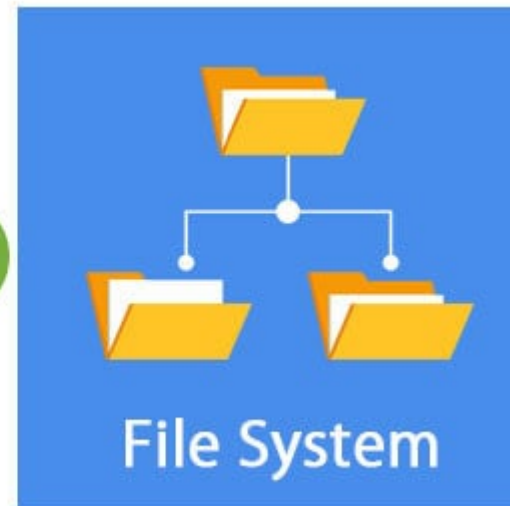
- **Data:** 200, 250, 300 (sales numbers for 3 months)
- **Information:** Sales are increasing 25% each month.
- **Knowledge:** The sales strategy is working — keep investing in marketing to maintain growth.

Why Not Just Use Files?

- ▶ Traditional file systems store data in separate files
- ▶ No standard access methods
- ▶ Difficult to share across applications



VS



Problems with File-based Systems

- ▶ **Data redundancy – Storing Same information multiple times.**
- ▶ **Inconsistency – Mismatch same data in different places.**
- ▶ **Security issues – Data Security**
- ▶ **No concurrency control – Multiple users accessing data at the same time.**



Introduction to DBMS

What is a Database?

In computing, a database is an organized collection of data or a type of data store



Wikipedia

What is a DBMS?

A Database Management System (DBMS) is software that allows users to create, manage, and access databases.

It acts as an intermediary between users and databases, providing tools for Storing, retrieving, updating and securing data.

What is a DBMS?



Types of Databases

- ▶ **Relational Databases (RDBMS) – Oracle, SQL Server etc...**
- ▶ **NoSQL – MongoDB, Cassandra etc...**
- ▶ **Cloud DBs: Firebase, AWS RDS etc...**
- ▶ **Specialized Databases – SQLite, Apache Hadoop**

Functions of DBMS

- ▶ **Storage management**
- ▶ **Query processing**
- ▶ **Transaction management**
- ▶ **Concurrency control**
- ▶ **Security & backup**

Advantages of DBMS

- ▶ **Reduced redundancy**
- ▶ **Data consistency**
- ▶ **Better security**
- ▶ **Multi-user access**
- ▶ **Backup & recovery**



Database Users and Environment

Types of Database Users

- ▶ **End Users** - People using the database system – both for maintaining the databases, as well as common users.
- ▶ **Application Programmers** - Responsible for implementing the specifications into programs
- ▶ **Database Designers** - Responsible for identifying the data to be stored in the database, and for choosing appropriate structures to represent and store this data.
- ▶ **Database Administrators (DBA)** - Responsible for authorizing access to the database, coordinating and monitoring its use, and acquiring software and hardware resources, Ensuring Backup and Recovery.

Database System Environment

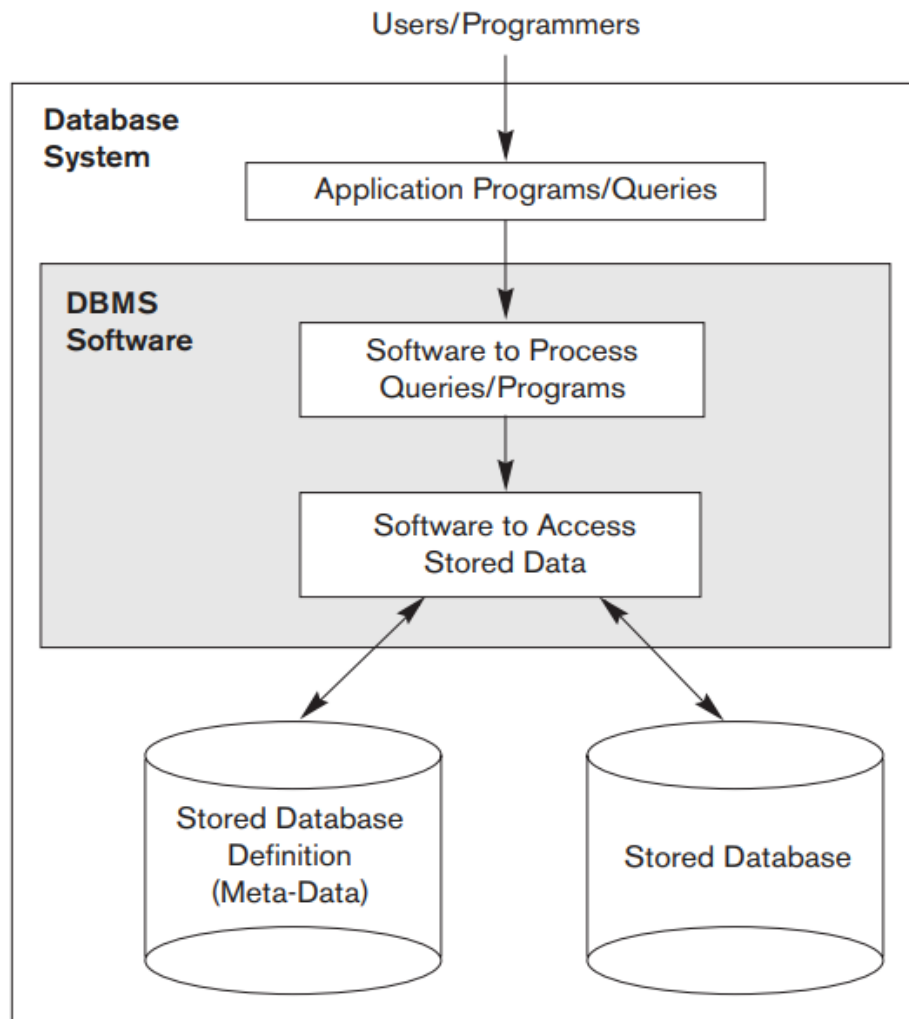


Figure 1.1
A simplified database system environment.

Database System Environment

- ▶ **Hardware** - Physical devices where the database runs. Includes: Servers, storage devices (HDD/SSD), networking equipment, and client machines.
- ▶ **Software** - DBMS, Operating System, Application Programs, Utilities & Tools.
- ▶ **Data** - The most important component. Can be operational data (daily transactions), metadata (data about data, e.g., schema), and historical data (archived).
- ▶ **Users** - ??
- ▶ **Procedures** - Policies and rules for using and managing the database. Examples: Backup schedules, security protocols, data access policies.

Real-Life Applications of Databases





Database Architecture

DBMS Architecture Overview

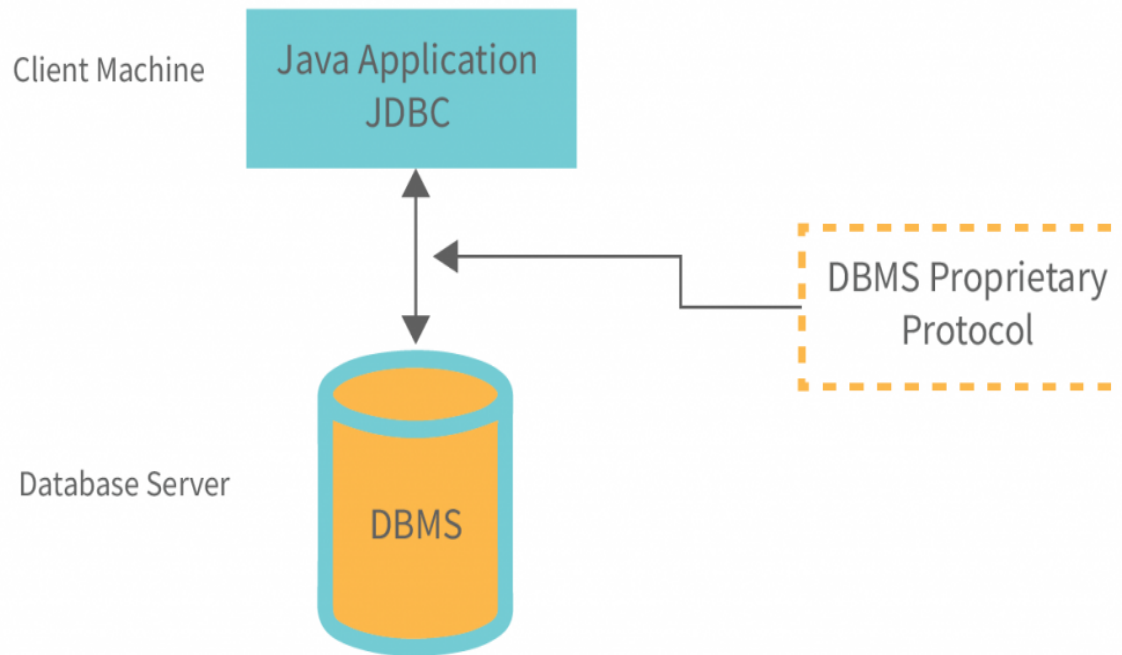
- ▶ **A DBMS (Database Management System) Architecture shows how data is stored, managed, and accessed in a database system.**
- ▶ **External Architecture - defines the structure and interaction between users, applications, and the DBMS.**
- ▶ **Internal Working Layers – defines the 3 Schema Architecture i.e. the levels of Data views.**

Two-Level Architecture

- ▶ **Application is divided into 2 layers:**
 - ▶ **Front-end (Client) – User interface – Sens Request**
 - ▶ **Back-end (Server) – DBMS Processes Queries, return results.**
- ▶ **Example: Java/Python app connecting to MySQL or SQL Server.**

Two-Level Architecture

Two-Tier Architecture

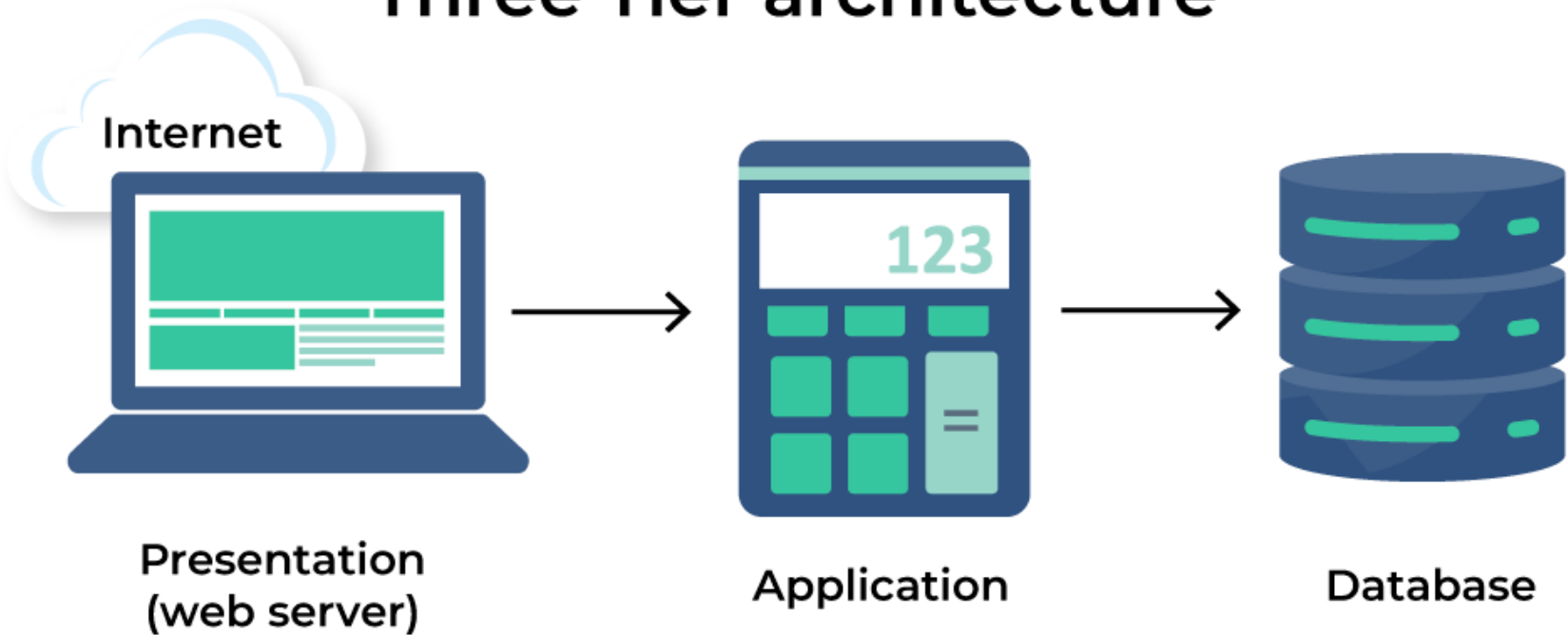


Three-Level Architecture

- ▶ **Presentation Layer (Client) – User interface (browser, app).**
- ▶ **Application Layer (Server/Business Logic) – Processes client requests, enforces rules.**
- ▶ **Database Layer (DBMS) – Stores, retrieves, manages data.**
- ▶ **Example: Web application React JS → PHP/Java/.NET middleware → Oracle/MySQL DB.**

Three-Level Architecture

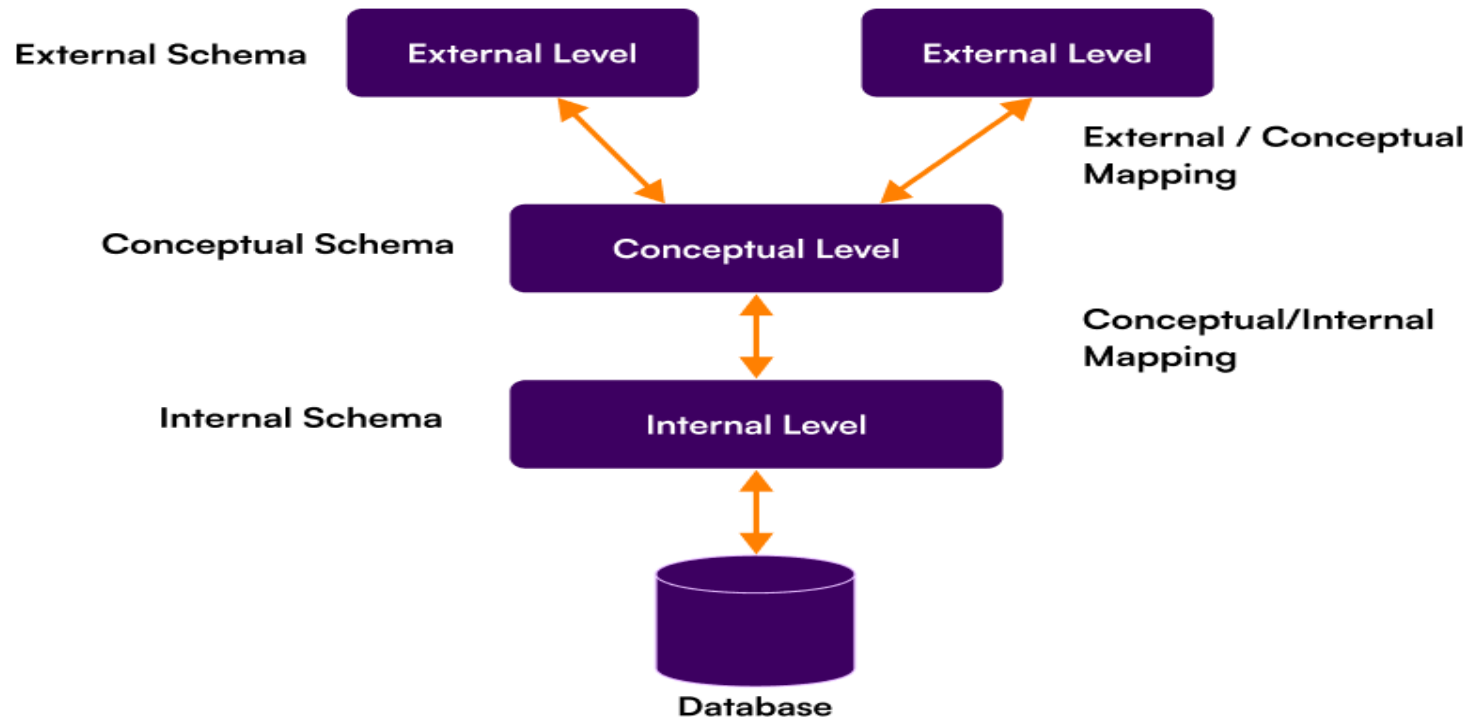
Three Tier architecture



Internal Working Layers

- ▶ **External Level (User View)**
 - ▶ Different users see different views of the database.
 - ▶ Example: Accountant sees salary data, HR sees employee info.
- ▶ **Conceptual Level (Logical View)**
 - ▶ Overall logical structure of the database (entities, relationships, schema).
- ▶ **Internal Level (Physical View)**
 - ▶ How data is physically stored on disks (indexes, file structure).

Internal Working Layers



Data Abstraction

- ▶ Data Abstraction means hiding unnecessary details from the user and showing only the essential information.
- ▶ It helps users interact with databases without worrying about how data is stored internally.
- ▶ Example:
 - ▶ A bank customer only sees account balance. They don't see how the DBMS stores it in files, indexes, or blocks on disk.

Internal Schema (Physical)

- ▶ Deals with how data is actually stored in the system.
- ▶ Includes storage structures, file formats, indexes, and pointers.
- ▶ Example: Data stored as blocks, bytes, or B-trees on a disk.
- ▶ Audience: Database administrators.

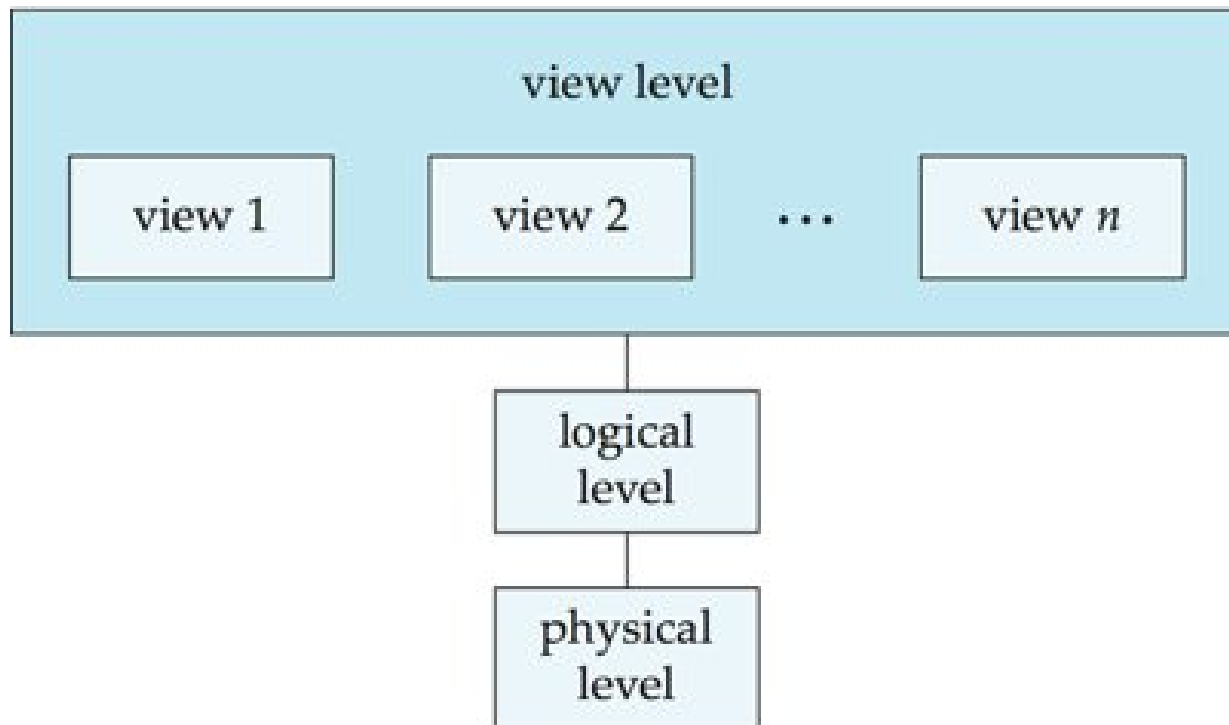
Conceptual Schema

- ▶ **Describes the structure of the database (entities, relationships, schema).**
- ▶ **Hides physical details but shows what data is stored and relationships.**
- ▶ **Example: Student table with attributes (ID, Name, Age, Course).**
- ▶ **Audience: Database designers.**

External Schema

- ▶ Provides different views for different users.
- ▶ Users only see relevant data according to their needs.
- ▶ Data Views
- ▶ Example:
 - ▶ A student sees only their grades.
 - ▶ A teacher sees grades for all students.
 - ▶ An admin sees fee + attendance + grades.
- ▶ Audience: End users.

Abstraction Layers





Core Database Concepts

Schema

- ▶ The overall design / structure of a database.
- ▶ Describes how data is organized and the relationships between data.
- ▶ Defined at the time of database creation and rarely changes.
- ▶ Like a blueprint of the database.
- ▶ Example:
 - In a Student database →
 - ▶ STUDENT (Student_ID, Name, Age, Course)
- ▶ This definition (table name + attributes) is the schema.

Instance

- ▶ The actual data stored in the database at a specific point in time.
- ▶ Can change frequently as data is inserted, updated, or deleted.
- ▶ Like a snapshot of the database at a given moment.
- ▶ Example:
 - Data inside the Student table:
 - ▶ (101, Ali, 20, CS101) (102, Sara, 22, CS201) (103, Bilal, 21, CS101)
- ▶ These rows are the instance of the database.

Schema vs Instance

Comparison Table

Feature	Schema (Structure)	Instance (Data)
Definition	Logical design / blueprint	Snapshot of data at a given time
Nature	Changes rarely	Changes frequently
Example	STUDENT(ID, Name, Age, Course)	(101, Ali, 20, CS101), (102, Sara, 22...)
Analogy	A building plan (blueprint)	The people currently living inside

Database Languages

- ▶ **SQL – Structured Query Language (ISO Standard – SQL:2019)**
- ▶ **DDL: Define schema**
- ▶ **DML: Manipulate data**
- ▶ **DCL: Control access**

Transactions in DBMS

- ▶ A transaction is a single logical unit of work performed on the database.
It may consist of one or more SQL operations (like INSERT, UPDATE, DELETE, SELECT).
- ▶ Example: Transferring money between two bank accounts:
 - ▶ Deduct ₹1000 from Account A
 - ▶ Add ₹1000 to Account B
- ▶ Both steps together form one transaction — either both happen, or neither happens.

Properties of Transaction (ACID)

- ▶ **Atomicity** – All operations in a transaction are executed completely or not at all.
 - ▶ Example: If money is deducted from A but not added to B, the transaction fails.
- ▶ **Consistency** – A transaction brings the database from one valid state to another valid state.
 - ▶ Example: Total balance before and after transfer remains the same.
- ▶ **Isolation** – Multiple transactions running at the same time should not interfere with each other.
 - ▶ Example: Two people transferring money simultaneously shouldn't affect each other's results.
- ▶ **Durability** – Once a transaction is committed, the changes remain even if the system crashes.
 - ▶ Example: After money transfer, data is safe even if the server restarts.

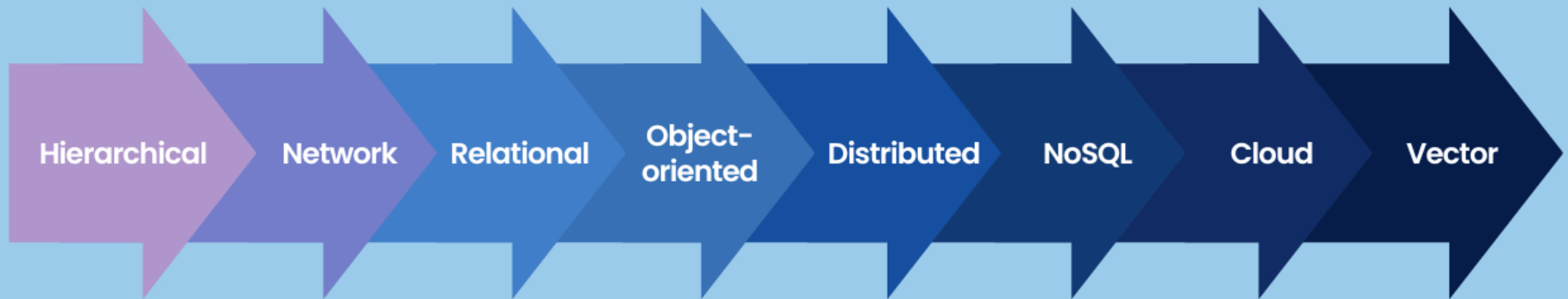
Indexing in DBMS

- ▶ Indexing is a technique to speed up data retrieval by creating small lookup tables that point to the actual data.
- ▶ Without an index: DBMS scans the entire table (slow).
- ▶ With an index: DBMS directly jumps to the location (fast).



History & Modern Trends

Evolution of Database Systems

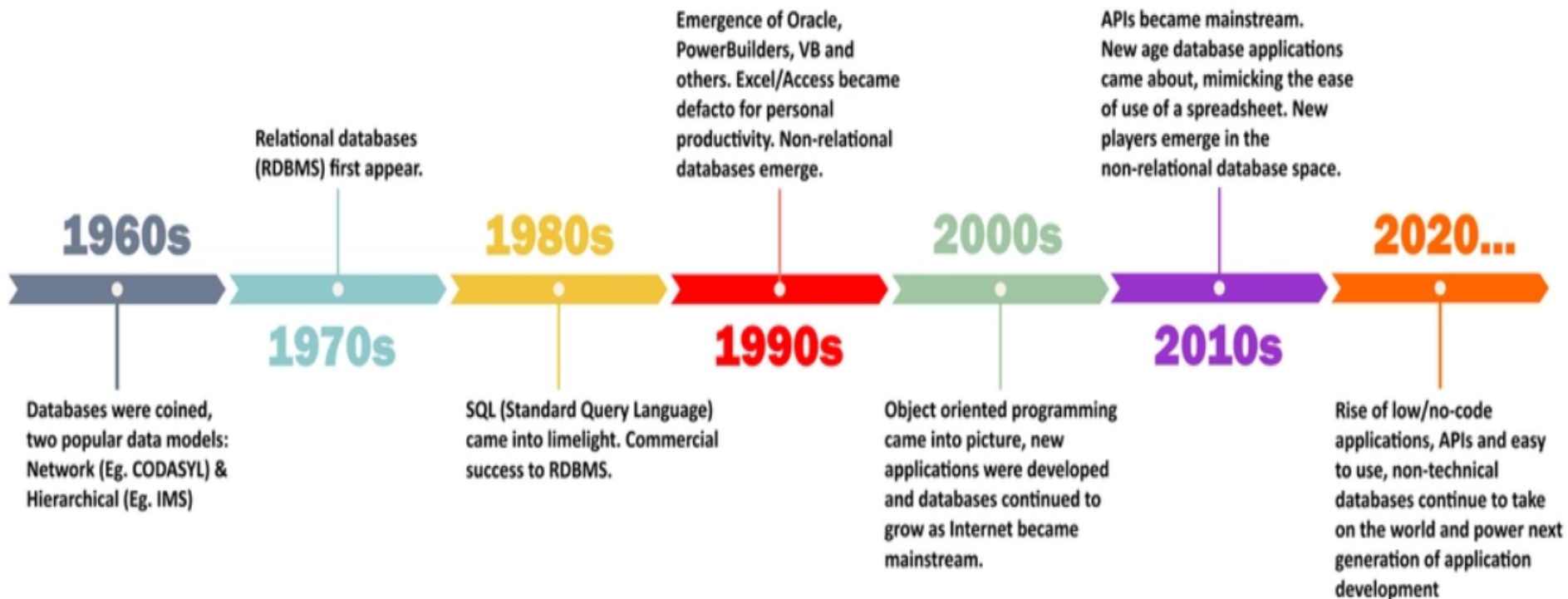


Evolution of Database Management Systems

STRATO
FLOW

Evolution of Database Systems

History of Databases (1960-2020)



Modern Databases

- ▶ **Distributed Databases**
- ▶ **Cloud Databases**
- ▶ **Big Data Technologies**



Example

EXAMPLE: A UNIVERSITY DATABASE

- ▶ Remember, DBMS is a general-purpose software system that facilitates the process of *defining, constructing, manipulating, sharing, protecting* and *maintaining* databases

EXAMPLE: A UNIVERSITY DATABASE

- ▶ What are the different relations you would need in a university database?
- ▶ How many columns will each relation have?
- ▶ What are the data restrictions?
- ▶ All of the above is defined by its meta-data

EXAMPLE: A UNIVERSITY DATABASE

- ▶ In order to come up with the database, it must start with the first step, which is ***defining a database***
- ▶ Here, all the specifications are collected about which files would we need in our database and how each of these files will be defined, i.e. its ***meta-data***

RELATIONS

Relation_name	No_of_columns
STUDENT	4
COURSE	4
SECTION	5
GRADE_REPORT	3
PREREQUISITE	2

COLUMNS

Column_name	Data_type	Belongs_to_relation
Name	Character (30)	STUDENT
Student_number	Character (4)	STUDENT
Class	Integer (1)	STUDENT
Major	Major_type	STUDENT
Course_name	Character (10)	COURSE
Course_number	XXXXNNNN	COURSE
....		
....		
....		
Prerequisite_number	XXXXNNNN	PREREQUISITE

Note: Major_type is defined as an enumerated type with all known majors.
XXXXNNNN is used to define a type with four alphabetic characters followed by four numeric digits.

EXAMPLE: A UNIVERSITY DATABASE

- ▶ The next step after defining the specifications is to ***construct the database***
- ▶ This requires storing data to represent each file as mentioned in the RELATIONS table of meta-data, i.e. STUDENT, COURSE, SECTION, etc.

STUDENT

Name	Student_number	Class	Major

COURSE

Course_name	Course_number	Credit_hours	Department

SECTION

Section_identifier	Course_number	Semester	Year	Instructor

GRADE_REPORT

Student_number	Section_identifier	Grade

PREREQUISITE

Course_number	Prerequisite_number

EXAMPLE: A UNIVERSITY DATABASE

- ▶ Then comes the step of *manipulating a database*
- ▶ This requires adding data to the files/tables

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	07	King
92	CS1310	Fall	07	Anderson
102	CS3320	Spring	08	Knuth
112	MATH2410	Fall	08	Chang
119	CS1310	Fall	08	Anderson
135	CS3380	Fall	08	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

EXAMPLE: A UNIVERSITY DATABASE

- ▶ Manipulation also involves retrieving data, updating it, deleting it, etc. using queries and transactions
- ▶ For example:
 - ▶ Queries to retrieve data:
 - ▶ Retrieve the transcript – a list of all courses and grades – of student named “Smith”
 - ▶ List the names of students who took the section of the “Database” course offered in Fall 2008 and their grades in the section
 - ▶ List the prerequisites of the “Database” course
 - ▶ Update database:
 - ▶ Change the class of student named “Smith” to Sophomore
 - ▶ Create a new section for the “Database” course for the current semester
 - ▶ Enter a grade “A” for student named “Smith” in the “Database” section of Fall 2008

EXAMPLE: A UNIVERSITY DATABASE

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	07	King
92	CS1310	Fall	07	Anderson
102	CS3320	Spring	08	Knuth
112	MATH2410	Fall	08	Chang
119	CS1310	Fall	08	Anderson
135	CS3380	Fall	08	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310



THANK YOU